Chapter 11. Analytics-on-write

This chapter covers

- Simple algorithms for analytics-on-write on event streams
- Modeling operational reporting as a DynamoDB table
- Writing an AWS Lambda function for analytics-on-write
- Deploying and testing an AWS Lambda function

In the previous chapter, we implemented a simple analytics-on-read strategy for OOPS, our fictitious package-delivery company, using Amazon Redshift. The focus was on storing our event stream in Redshift in such a way as to support as many analyses as possible "after the fact." We modeled a fat event table, widened it further with dimension lookups for key entities including drivers and trucks, and then tried out a few analyses on the data in SQL.

For the purposes of this chapter, we will assume that some time has passed at OOPS, during which the BI team has grown comfortable with writing SQL queries against the OOPS event stream as stored in Redshift. Meanwhile, rumblings are coming from various stakeholders at OOPS who want to see analyses that are not well suited to Redshift. For example, they are interested in the following:

- ***Low-latency operational reporting***— This must be fed from the incoming event streams in as close to real time as possible.
- ***Dashboards to support thousands of simultaneous users***— For example, a parcel tracker on the website for OOPS customers.

In this chapter, we will explore techniques for delivering these kinds of analytics, which broadly fall under the term *analytics-on-write*. Analytics-on-write has an up-front cost: we have to decide on the analysis we want to perform ahead of time and put this analysis live on our event stream. In return for this constraint, we get some benefits: our queries are low latency, can serve many simultaneous users, and are simple to operate.

To implement analytics-on-write for OOPS, we are going to need a database for storing our aggregates, and a stream processing framework to turn our events into aggregates. For the database, we will use Amazon DynamoDB, which is a hosted, highly scalable key-value store with a relatively simple query API. For the stream processing piece, we are going to try out AWS Lambda, which is an innovative platform for single-event processing, again fully hosted by Amazon Web Services.

Let's get started!

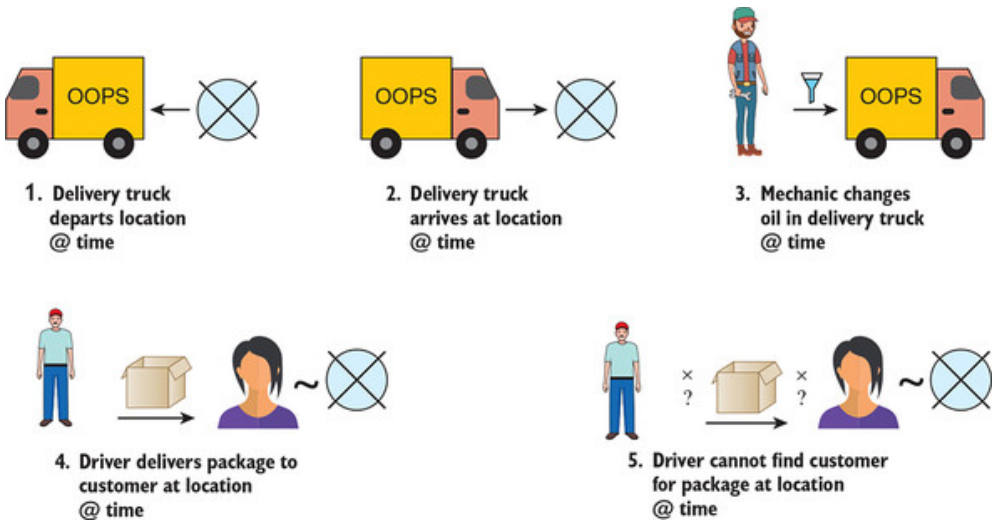
11.1. Back to OOPS

For this chapter, we will be returning to OOPS, the scene of our analytics-on-read victory, to implement analytics-on-write. Before we can get started, we need to get Kinesis set up and find out exactly what analytics our OOPS bosses are expecting.

11.1.1. Kinesis setup

In the preceding chapter, we interacted with the OOPS unified log through an archive of events stored in Amazon S3. This archive contained the five types of OOPS events, all stored in JSON, as recapped in figure 11.1. This event archive suited our analytics-on-read requirements fine, but for analytics-on-write, we need something much "fresher": we need access to the OOPS event stream as it flows through Amazon Kinesis in near real-time.

Figure 11.1. The five types of events generated at OOPS are unchanged since their introduction in the previous chapter.



Our OOPS colleagues have not yet given us access to the "firehose" of live events in Kinesis, but they have shared a Python script that can generate valid OOPS events and write them directly to a Kinesis stream for testing. You can find this script in the GitHub repository:

```
ch11/11.1/generate.py
```

Before we can use this script, we must first set up a new Kinesis stream to write the events to. As in chapter 4, we can do this by using the AWS CLI tools:

```
$ aws kinesis create-stream --stream-name oops-events \
  --shard-count 2 --region=us-east-1 --profile=ulp
```

That command doesn't give any output, so let's follow up by describing the stream:

```
$ aws kinesis describe-stream --stream-name oops-events \
  --region=us-east-1 --profile=ulp
{
    "StreamDescription": {
        "StreamStatus": "ACTIVE",
        "StreamName": "oops-events",
        "StreamARN": "arn:aws:kinesis:us-east-1:719197435995:stream/
➥ oops-events",
...
```

We are going to need the stream's ARN later, so let's add it into an environment variable:

```
$ stream_arn=arn:aws:kinesis:us-east-1:719197435995:stream/oops-events
```

Now let's try out the event generator:

```
$ /vagrant/ch11/11.1/generate.py
Wrote TruckArrivesEvent with timestamp 2018-01-01 00:14:00
Wrote DriverMissesCustomer with timestamp 2018-01-01 02:14:00
Wrote DriverDeliversPackage with timestamp 2018-01-01 04:03:00
```

Great—we can successfully write events to our Kinesis stream. Press Ctrl-C to cancel the generator.

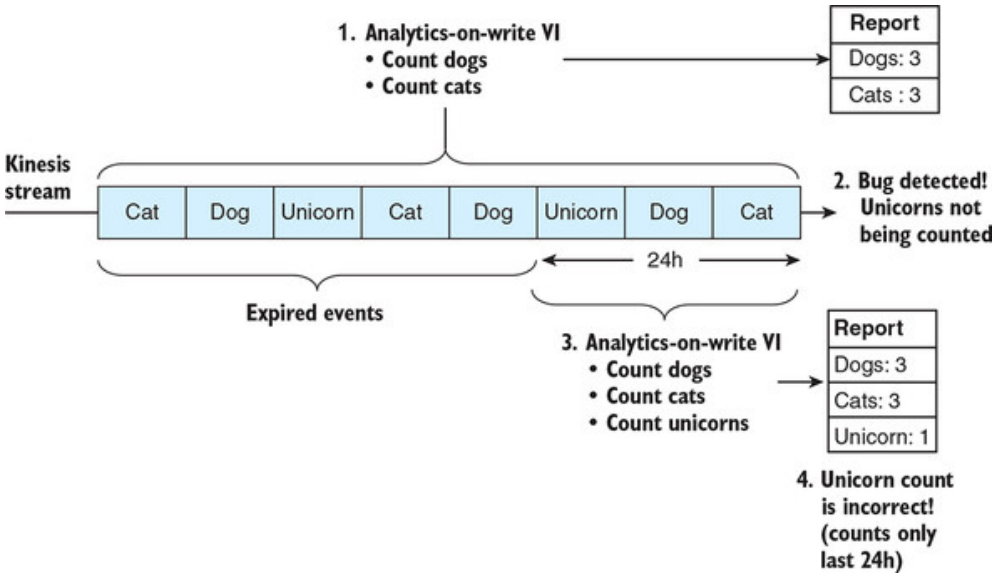
11.1.2. Requirements gathering

With analytics-on-read, our focus was on devising the most flexible way of storing our event stream in Redshift, so that we could support as great a variety of *post facto* analyses as possible. The structure of our event storage was optimized for future flexibility, rather than being tied to any specific analysis that OOPS might want to perform later.

With analytics-on-write, the opposite is true: we must understand *exactly* what analysis OOPS wants to see, so that we can build that analysis to run in near real-time on our Kinesis event stream. And, ideally, we would get this right the first time; remember that a Kinesis stream stores only the last 24 hours of events (configurable up to one week), after which the events auto-expire. If there are any mistakes in our analytics-on-write

processing, at best we will be able to rerun for only the last 24 hours (or one week). This is visualized in figure 11.2.

Figure 11.2. In the case of a bug in our analytics-on-write implementation, we have to fix the bug and redeploy the analytics against our event stream. At best, we can recover the last 24 hours of missing data, as depicted here. At worst, our output is corrupted, and we have to restart our analytics from scratch.



This means that we need to sit down with the bigwigs at OOPS and find out exactly what near-real-time analytics they want to see. When we do this, we learn that the priority is around operational reporting about the delivery trucks. In particular, OOPS wants a near-real-time feed that tells them the following:

- The location of each delivery truck
- The number of miles each delivery truck has driven since its last oil change

This sounds like a straightforward analysis to generate. We can sketch out a simple table structure that holds all of the relevant data, as shown in table 11.1.

Table 11.1. The status of OOPS's delivery trucks

Truck VIN	Latitude	Longitude	Miles since oil change
1HGCM82633A004352	51.5208046	-0.1592323	35
JH4TB2H26CC000000	51.4972997	-0.0955459	167

Truck VIN	Latitude	Longitude	Miles since oil change
19UYA31581L000000	51.4704679	-0.1176902	78

Now we have our five event types flowing into Kinesis, and we understand the analysis that our OOPS coworkers are looking for. In the next section, let's define the analytics-on-write algorithm that will connect the dots.

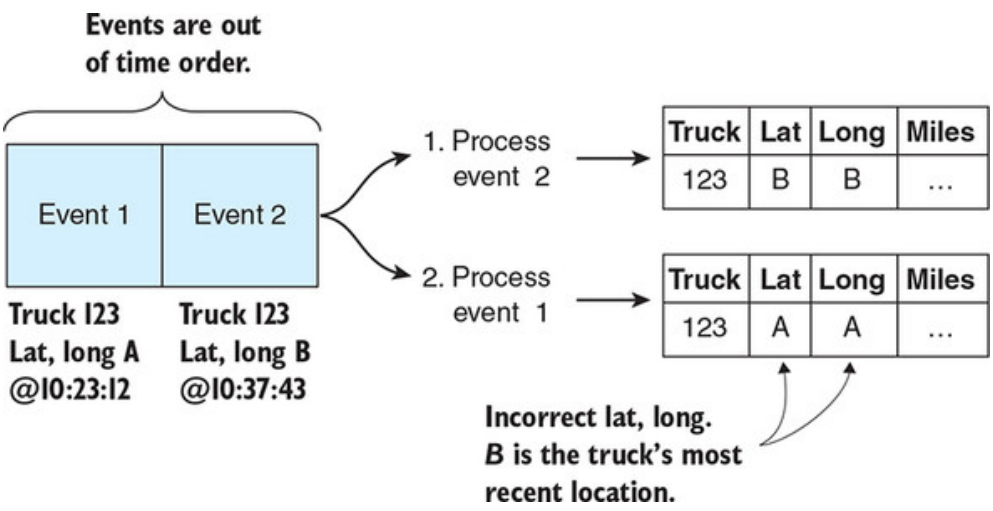
11.1.3. Our analytics-on-write algorithm

We need to create an algorithm that will use the OOPS event stream to populate our four-column table and keep it up-to-date. Broadly, this algorithm needs to do the following:

1. Read each event from Kinesis.
2. If the event involves a delivery truck, use its current mileage to update the count of miles since the last oil change.
3. If the event is an oil change, reset the count of miles since the last oil change.
4. If the event associates a delivery truck with a specific location, update the table row for the given truck with the event's latitude and longitude.

Our approach will have similarities to the *stateful event processing* that we implemented in chapter 5. In this case, we are not interested in processing multiple events at a time, but we do need to make sure that we are always updating our table with a truck's most recent location. There is a risk that if our algorithm ingests an older event after a newer event, it could incorrectly overwrite the "latest" latitude and longitude with the older one. This danger is visualized in figure 11.3.

Figure 11.3. If an older event is processed after a newer event, a naïve algorithm could accidentally update our truck's "current" location with the older event's location.



We can prevent this by storing an additional piece of state in our table, as shown in table 11.2. The new column, *Location timestamp*, records the timestamp of the event from which we have taken each truck's current latitude and longitude. Now when we process an event from Kinesis containing a truck's location, we check this event's timestamp against the existing Location timestamp in our table. Only if the new event's timestamp "beats" the existing timestamp (it's more recent) do we update our truck's latitude and longitude.

Table 11.2. Adding the *Location timestamp* column to our table

Truck VIN	Latitude	Longitude	Location timestamp	Miles since oil change
1HGCM8...	51.5208046	-0.1592323	2018-08-02T21:50:49Z	35
JH4TB2...	51.4972997	-0.0955459	2018-08-01T22:46:12Z	167
19UYA3...	51.4704679	-0.1176902	2018-08-02T18:14:45Z	78

This leaves only our last metric, *Miles since oil change*. This one is slightly more complicated to calculate. The trick is to realize that we should not be storing this metric in our table. Instead, we should be storing the two inputs that go into calculating this metric:

- Current (latest) mileage
- Mileage at the time of the last oil change

With these two metrics stored in our table, we can calculate *Miles since oil change* whenever we need it, like so:

```
miles since oil change = current mileage – mileage at last oil change
```

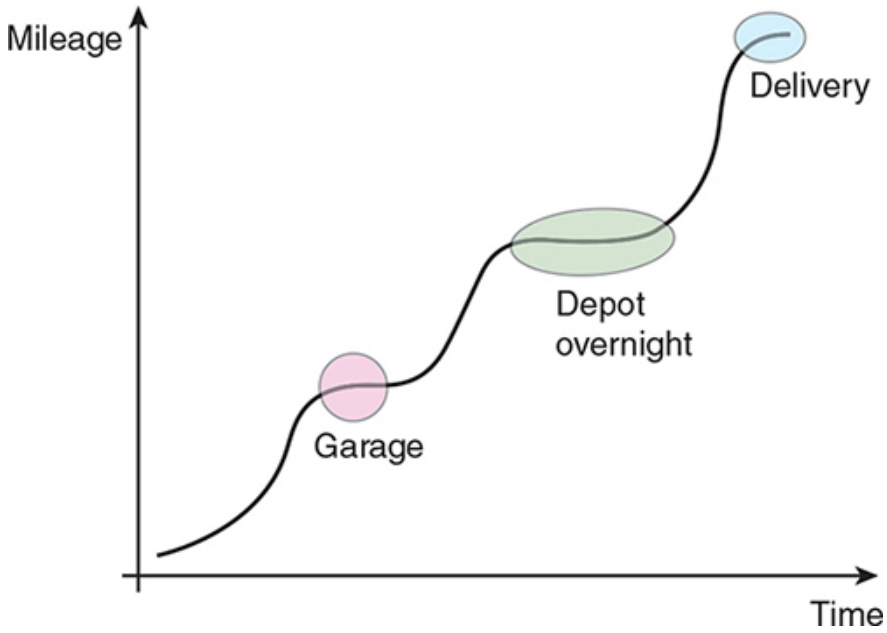
For the cost of a simple calculation at serving time, we have a much easier-to-maintain table, as you will see in the next section. Table 11.3 shows the final version of our table structure, with the new *Mileage* and *Mileage at oil change* columns.

Table 11.3. Replacing our *Miles since oil change* metric with *Mileage* and *Mileage at oil change*

Truck VIN	Latitude	Longitude	Location timestamp	Mileage	Mileage at oil change
1HGCM8...	51.5...	-0.15...	2018-08-02T21:50:49Z	12453	12418
JH4TB2...	51.4...	-0.09...	2018-08-01T22:46:12Z	19090	18923
19UYA3...	51.4...	-0.11...	2018-08-02T18:14:45Z	8407	8329

Remember that for this table to be accurate, we always want to be recording the truck's most recent mileage, and the truck's mileage at its last-known oil change. Again, how do we handle the situation where an *older* event arrives after a *more recent* event? This time, we have something in our favor, which is that a truck's mileage *monotonically increases* over time, as per figure 11.4. Given two mileages for the same truck, the higher mileage is always the more recent one, and we can use this rule to unambiguously discard the mileages of older events.

Figure 11.4. The mileage recorded on a given truck's odometer monotonically increases over time. We can see periods where the mileage is flat, but it never decreases as time progresses.



Putting all this together, we can now fully specify our analytics-on-write algorithm in pseudocode. First of all, we know that all of our analytics relate to trucks, so we should start with a filter that excludes any events that don't include a vehicle entity:

```
let e = event to process
if e.event not in ("TRUCK_ARRIVES", "TRUCK_DEPARTS",
  "MECHANIC_CHANGES_OIL") then
  skip event
end if
```

Now let's process our vehicle's mileage:

```
let vin = e.vehicle.vin
let event_mi = e.vehicle.mileage
let current_mi = table[vin].mileage
if current_mi is null or current_mi < event_mi then
    set table[vin].mileage = event_mi
end if
```

Most of this pseudocode should be self-explanatory. The trickiest part is the `table[vin] .mileage` syntax, which references the value of a column for a given truck in our table. The important thing to understand here is that we update a truck's mileage in our table only if the event being processed reports a *higher* mileage than the one already in our table for this truck.

Let's move on our truck's location, which is captured only in events relating to a truck arriving or departing:

```
if e.event == "TRUCK_ARRIVES" or "TRUCK_DEPARTS" then
    let ts = e.timestamp
```

```
        if ts > table[vin].location_timestamp then
            set table[vin].location_timestamp = ts
            set table[vin].latitude = e.location.latitude
            set table[vin].longitude = e.location.longitude
        end if
    end if
```

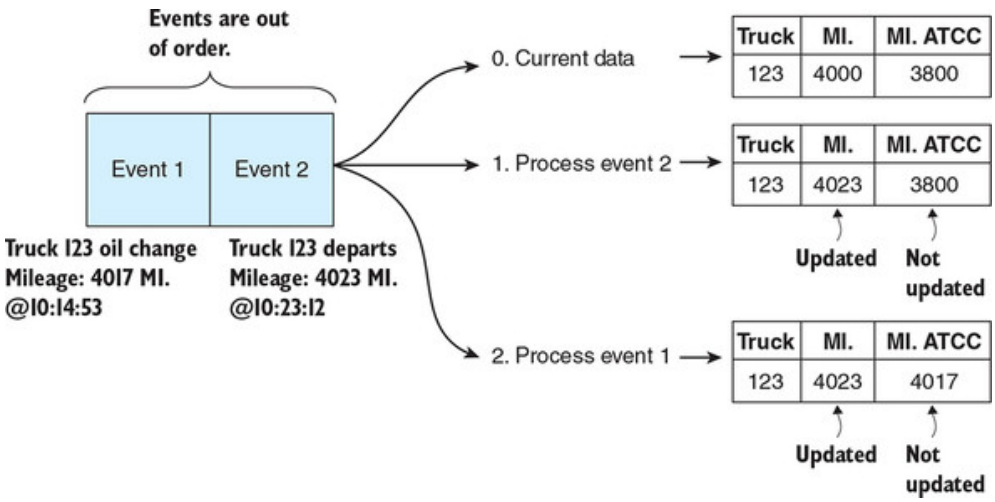
The logic here is fairly simple. Again, we have an `if` statement that makes updating the location conditional on the event's timestamp being newer than the one currently found in our table. This is to ensure that we don't accidentally overwrite our table with a stale location.

Finally, we need to handle any oil-change events:

```
if e.event == "MECHANIC_CHANGES_OIL" then
    let current_maoc = table[vin].mileage_at_oil_change
    if current_maoc is null or event_mi > current_maoc then
        set table[vin].mileage_at_oil_change = event_mi
    end if
end if
```

Again, we have a "guard" in the form of an `if` statement, which makes sure that we update the *Mileage at oil change* for this truck only if this oil-change event is newer than any event previously fed into the table. Putting it all together, we can see that this guard-heavy approach is safe even in the case of out-of-order events, such as when an oil-change event is processed *after* a *subsequent* truck-departs event. This is shown in figure 11.5.

Figure 11.5. The `if` statements in our analytics-on-write algorithm protect us from out-of-order events inadvertently overwriting current metrics in our table with stale metrics.



If OOPS were a real company, it would be crucial to get this algorithm reviewed and signed off before implementing the stream processing job.

With analytics-on-read, if we make a mistake in a query, we just cancel it and run the revised query. With analytics-on-write, a mistake in our algorithm could mean having to start over from scratch. Happily, in our case, we can move swiftly onto the implementation!

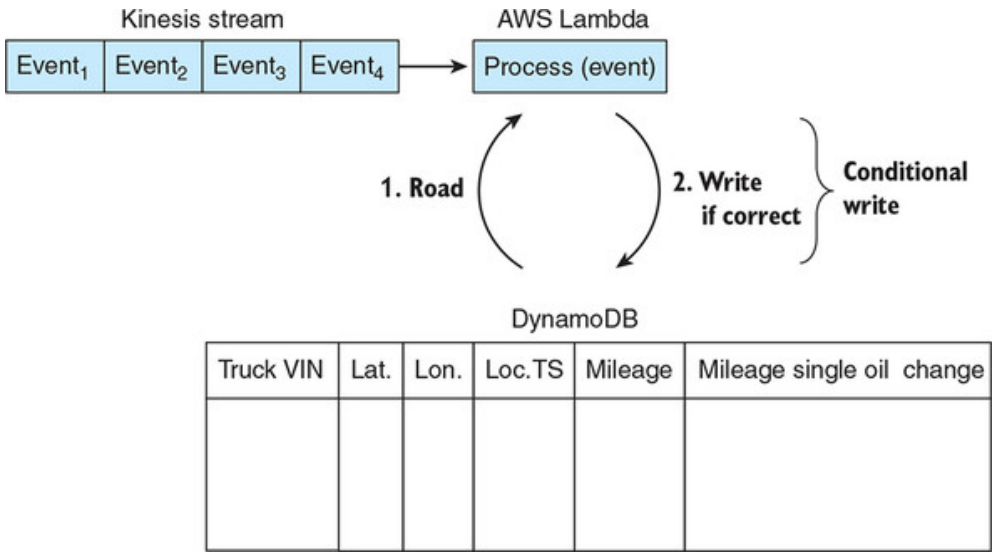
11.2. Building our Lambda function

Enough theory and pseudocode—in this section, we will implement our analytics-on-write by using AWS Lambda, populating our delivery-truck statuses to a table in DynamoDB. Let's get started.

11.2.1. Setting up DynamoDB

We are going to write a stream processing job that will read OOPS events, cross-check them against existing values in our table, and then update rows in that table accordingly. Figure 11.6 shows this flow.

Figure 11.6. Our AWS Lambda function will read individual events from OOPS, check whether our table in DynamoDB can be updated with the event's values, and update the row in DynamoDB if so. This approach uses a DynamoDB feature called conditional writes.



We already have a good idea of the algorithm we are going to implement, but rather than jumping into that, let's work backward from the table we need to populate. This table is going to live in Amazon's DynamoDB, a highly scalable hosted key-value store that is a great fit for analytics-on-write on AWS. We can create our table in DynamoDB by using the AWS CLI tools like so:

```
$ aws dynamodb create-table --table-name oops-trucks \
  --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 \
  --attribute-definitions AttributeName=vin,AttributeType=S \
  --key-schema AttributeName=vin,KeyType=HASH --profile=ulp \
  --region=us-east-1
{
```

```
    "TableDescription": {
        "TableArn": "arn:aws:dynamodb:us-east-1:719197435995:table/
➥ oops-trucks",
...
```

When creating a DynamoDB table, we have to specify only a few properties up-front:

- The `--provisioned-throughput` tells AWS how much throughput we want to reserve for reading and writing data from the table. For the purposes of this chapter, low values are fine for both.
- The `--attribute-definitions` defines an attribute for the vehicle identification number, which is a `string` called `vin`.
- The `--key-schema` specifies that the `vin` attribute will be our table's primary key.

Our table has now been created. We can now move on to creating our AWS Lambda function.

11.2.2. Introduction to AWS Lambda

The central idea of AWS Lambda is that developers should be writing *functions*, not *servers*. With Lambda, we write self-contained functions to process events, and then we publish those functions to Lambda to run. We don't worry about developing, deploying, or managing servers. Instead, Lambda takes care of scaling out our functions to meet the incoming event volumes. Of course, Lambda functions run on servers under the hood, but those servers are abstracted away from us.

At the time of writing, we can write our Lambda functions to run either on Node.js or on the JVM. In both cases, the function has a similar API, in pseudocode:

```
def recordHandler(events: List[Event])
```

This function signature has some interesting features:

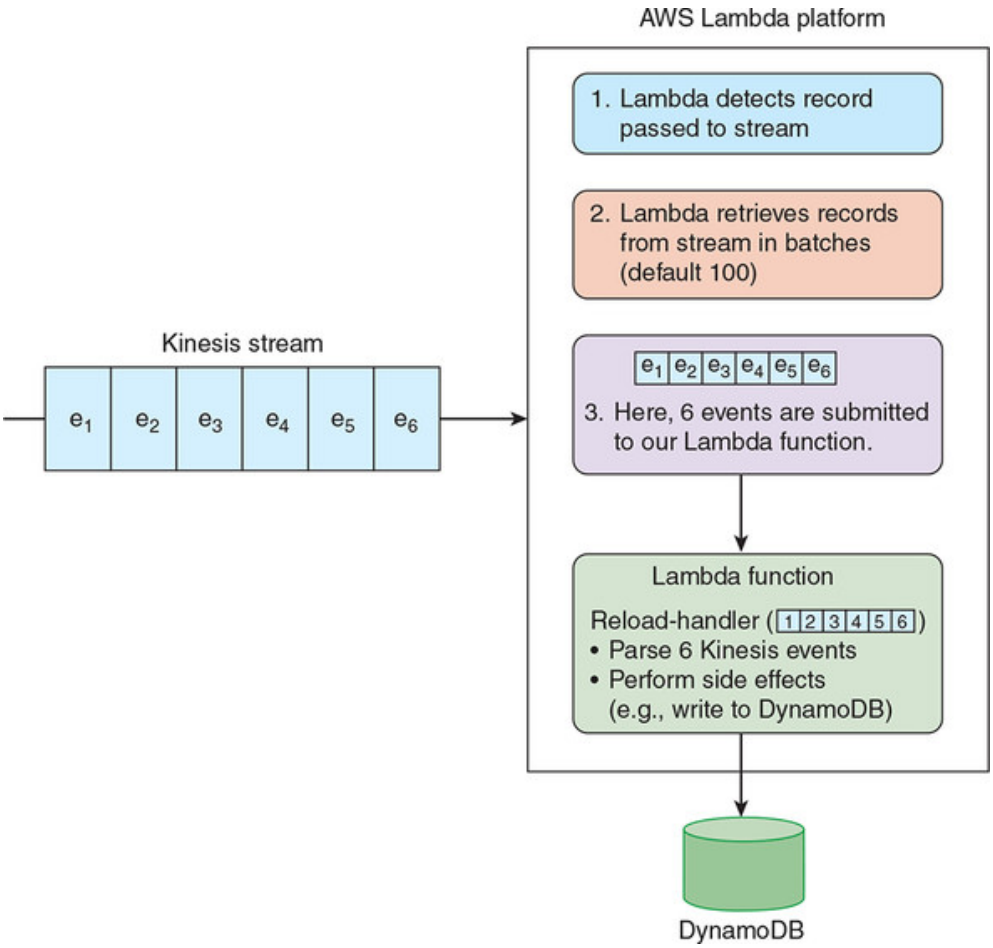
- The function operates on a list or array of events, rather than a single one. Under the hood, Lambda is collecting a *microbatch* of events before invoking the function.
- The function does not return anything. It exists only to produce *side effects*, such as writing to DynamoDB or creating new Kinesis events.

Figure 11.7 illustrates these two features of a Lambda function.

Does this function API look familiar? Remember back in chapter 5 when we wrote event-processing jobs using Apache Samza, the API looked like this:

```
public void process(IncomingMessageEnvelope envelope,
    MessageCollector collector, TaskCoordinator coordinator)
```

Figure 11.7. AWS Lambda detects records posted to a Kinesis stream, collects a microbatch of those records, and then submits that microbatch to the specified Lambda function.



In both cases, we are implementing a side-effecting function that is invoked for incoming events—or in Samza's case, a single event. And as a Lambda function is run on AWS Lambda, so our Samza jobs were run on Apache YARN: we can say that Lambda and YARN are the execution environments for our functions. The similarities and differences of Samza and Lambda are explored further in table 11.4; this table shows more similarities than we might expect.

Table 11.4. Comparing features of Apache Samza and AWS Lambda

Feature	Apache Samza	AWS Lambda

Feature	Apache Samza	AWS Lambda
Function invoked for	A single event	A microbatch of events
Executed on	Apache YARN (open source)	AWS Lambda (proprietary)
Events supported	Apache Kafka	Kafka and Kinesis records, S3 events, DynamoDB events, SNS notifications
Write function in	Java	JavaScript, Java 8, Scala (so far)
Store state	Locally or remotely	Remotely

We are going to write our Lambda function in Scala, which is supported for JVM-based Lambdas alongside Java. Let's get started.

11.2.3. Lambda setup and event modeling

We need to build our Lambda function as a single fat jar file that contains all of our dependencies and can be uploaded to the Lambda service. As before, we will use Scala Build Tool, with the `sbt-assembly` plugin to build our fat jar. Let's start by creating a folder to work in, ideally inside the book's Vagrant virtual machine:

```
$ mkdir ~/aow-lambda
```

Let's add a file in the root of this folder called build.sbt, with the contents as shown in the following listing.

Listing 11.1. build.sbt

```
javacOptions ++= Seq("-source", "1.8", "-target", "1.8", "-Xlint")

lazy val root = (project in file(".")).
  settings(
    name := "aow-lambda",
    version := "0.1.0",
    scalaVersion := "2.12.7",
    retrieveManaged := true,
    libraryDependencies += "com.amazonaws" % "aws-lambda-java-core" %
```

```
    ➥ "1.2.0",
      libraryDependencies += "com.amazonaws" % "aws-lambda-java-events" %
    ➥ "2.2.4",
      libraryDependencies += "com.amazonaws" % "aws-java-sdk" % "1.11.473"
    ➥ % "provided",
      libraryDependencies += "com.amazonaws" % "aws-java-sdk-core" %
    ➥ "1.11.473" % "provided",
      libraryDependencies += "com.amazonaws" % "aws-java-sdk-kinesis" %
    ➥ "1.11.473" % "compile",
      libraryDependencies += "com.fasterxml.jackson.module" %
    ➥ "jackson-module-scala_2.12" % "2.8.4",
      libraryDependencies += "org.json4s" %% "json4s-jackson" % "3.6.2",
      libraryDependencies += "org.json4s" %% "json4s-ext" % "3.6.2",
      libraryDependencies += "com.github.seratch" %% "awscala" % "0.8.+"
    )

mergeStrategy in assembly := {
    case PathList("META-INF", xs @ _*) => MergeStrategy.discard
    case x => MergeStrategy.first
}

jarName in assembly := { s"${name.value}-${version.value}" }
```

- ***1*** **Our Lambda function will be compiled against Scala 2.12.7 on Java 8.**
- ***2*** **"provided" libraries are available in Lambda and so do not need to be bundled in our fat jar; "compile" ones are bundled in the fat jar.**
- ***3*** **How to handle conflicts when merging the dependencies into our fat jar**

To handle the assembly of our fat jar, we need to create a project subfolder with a plugins.sbt file within it. Create it like so:

```
$ mkdir project
$ echo 'addSbtPlugin("com.eed3si9n" % "sbt-assembly" % "0.14.9")' > \
  project/plugins.sbt
```

If you are running this from the book's Vagrant virtual machine, you should have Java 8, Scala, and SBT installed already. Check that our build is set up correctly:

```
$ sbt assembly
...
[info] Packaging /vagrant/ch11/11.2/aow-lambda/target/scala-2.12/
➥ aow-lambda-0.1.0 ...
[info] Done packaging.
[success] Total time: 860 s, completed 20-Dec-2018 18:33:57
```

Now we can move on to our event-handling code. If you look back at our analytics-on-write algorithm in section 11.1.3, you'll see that the logic is largely driven by which OOPS event type we are currently processing. So, let's first write some code to deserialize our incoming event into an appropriate Scala case class. Create the following file:

```
src/main/scala/aowlambda/events.scala
```

This file should be populated with the contents of the following listing.

Listing 11.2. events.scala

```
package aowlambda

import java.util.UUID, org.joda.time.DateTime
import org.json4s._, org.json4s.jackson.JsonMethods._

case class EventSniffer(event: String)
case class Employee(id: UUID, jobRole: String)
case class Vehicle(vin: String, mileage: Int)
case class Location(latitude: Double, longitude: Double, elevation: Int)
case class Package(id: UUID)
case class Customer(id: UUID, isVip: Boolean)

sealed trait Event
case class TruckArrives(timestamp: DateTime, vehicle: Vehicle,
  location: Location) extends Event
case class TruckDeparts(timestamp: DateTime, vehicle: Vehicle,
  location: Location) extends Event
case class MechanicChangesOil(timestamp: DateTime, employee: Employee,
  vehicle: Vehicle) extends Event
case class DriverDeliversPackage(timestamp: DateTime, employee: Employee
  `package`: Package, customer: Customer, location: Location) extends Ev
case class DriverMissesCustomer(timestamp: DateTime, employee: Employee,
  `package`: Package, customer: Customer, location: Location) extends Ev

object Event {

  def fromBytes(byteArray: Array[Byte]): Event = {
    implicit val formats = DefaultFormats ++ ext.JodaTimeSerializers.all
      ext.JavaTypesSerializers.all
    val raw = parse(new String(byteArray, "UTF-8"))
    raw.extract[EventSniffer].event match {
      case "TRUCK_ARRIVES" => raw.extract[TruckArrives]
      case "TRUCK_DEPARTS" => raw.extract[TruckDeparts]
      case "MECHANIC_CHANGES_OIL" => raw.extract[MechanicChangesOil]
      case "DRIVER_DELIVERS_PACKAGE" => raw.extract[DriverDeliversPackag
      case "DRIVER_MISSES_CUSTOMER" => raw.extract[DriverMissesCustomer]
      case e => throw new RuntimeException("Didn't expect " + e)
    }
```

```
    }
}
```

- ***1*** **Contains just the event's type, so we can determine which case class to deserialize into**
- ***2*** **Representing our events as an algebraic data type (ADT)**
- ***3*** **Pattern match on the event's type to construct the final Event.**
- ***4*** **In case we have an unexpected event type**

Let's use the Scala console or REPL to check that this code is working correctly:

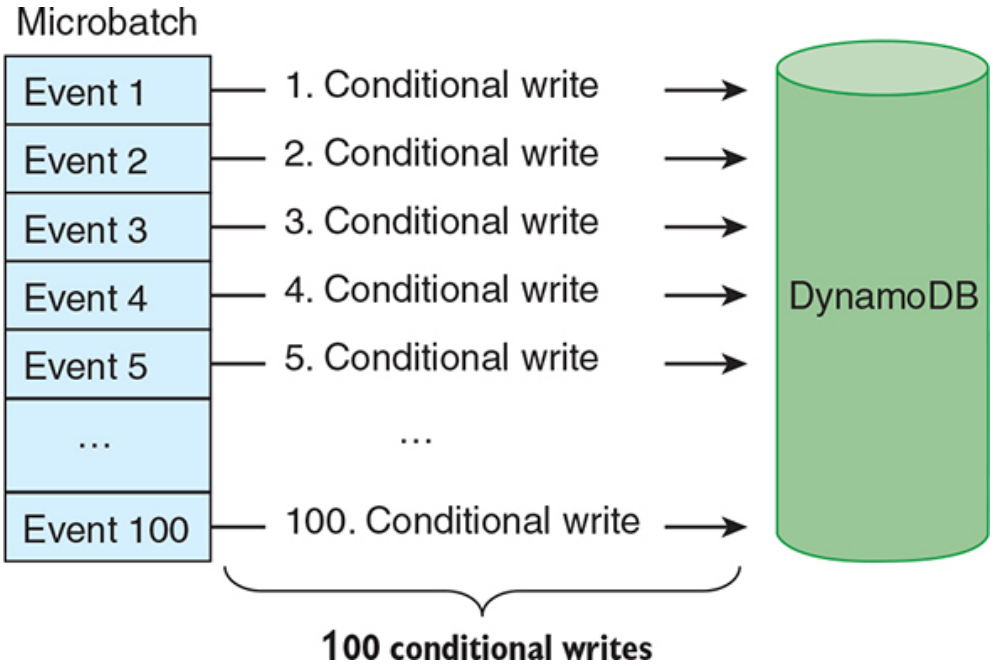
```
$ sbt console
scala> val bytes = """{"event":"TRUCK_ARRIVES", "location": {"elevation"
  "latitude":51.522834, "longitude":-0.081813},
  "timestamp": "2018-01-12T12:42:00Z", "vehicle": {"mileage":33207,
  "vin":"1HGCM82633A004352"}}""".getBytes("UTF-8")
bytes: Array[Byte] = Array(123, ...
scala> aowlambda.Event.fromBytes(bytes)
res0: aowlambda.Event = TruckArrivesEvent(2018-01-12T12:42:00.000Z,
 Vehicle(1HGCM82633A004352,33207),Location(51.522834,-0.081813,7))
```

Great—we can see that a byte array representing a *Truck arrives* event in JSON format is being correctly deserialized into our `TruckArrivesEvent` case class in Scala. Now we can move onto implementing our analytics-on-write algorithm.

11.2.4. Revisiting our analytics-on-write algorithm

Section 11.1.3 laid out the algorithm for our OOPS analytics-on-write by using simple pseudocode. This pseudocode was intended to process a single event at a time, but we have since learned that our Lambda function will be given a microbatch of up to 100 events at a time. Of course, we could apply our algorithm to each event in the microbatch, as shown in figure 11.8.

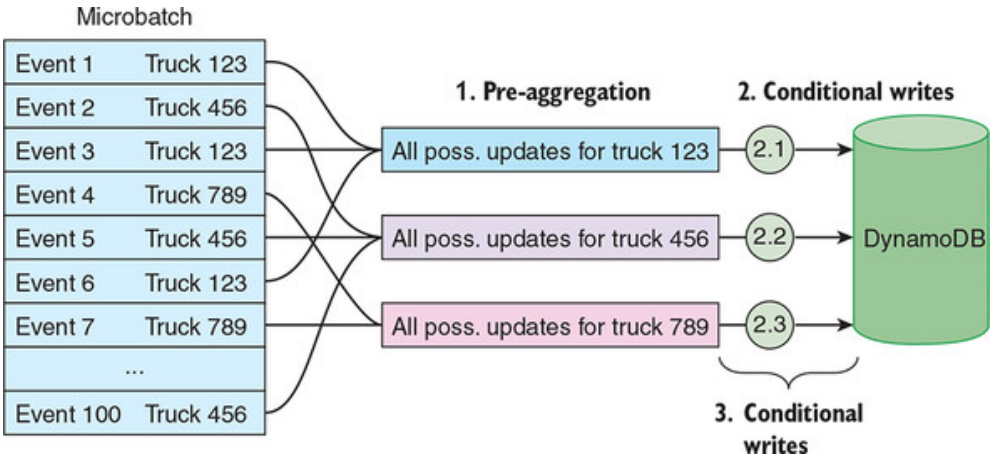
Figure 11.8. A naïve approach to updating our DynamoDB table from our microbatch of events would involve a conditional write for every single event.



Unfortunately, this approach is wasteful whenever a single microbatch contains multiple events relating to the same OOPS truck, as could easily happen. Remember how we want to update our DynamoDB table with a given truck's most recent (highest) mileage? If our microbatch contains 10 events relating to Truck 123, the naïve implementation of our algorithm from figure 11.8 would require 10 conditional writes to the DynamoDB table just to update the Truck 123 row. Reading and writing to remote databases is an expensive operation, so it's something that we should aim to minimize in our Lambda function, especially given that each Lambda function call has to complete within 60 seconds (configurable up to 15 minutes).

The solution is to *pre-aggregate* our microbatch of events inside our Lambda function. With 10 events in the microbatch relating to Truck 123, our first step should be to find the highest mileage across these 10 events. This highest mileage figure is the only one that we need to attempt to write to DynamoDB; there is no point bothering DynamoDB with the nine lower mileages at all. Figure 11.9 depicts this pre-aggregation technique.

Figure 11.9. By applying a pre-aggregation to our microbatch of events, we can reduce the required number of conditional writes down to one per OOPS truck found in our microbatch.



Of course, the pre-aggregation step may not always bear fruit. For example, if all 100 events in the microbatch relate to different OOPS trucks, there is nothing to reduce. This said, the pre-aggregation is a relatively cheap in-memory process, so it's worth always attempting it, even if we prevent only a handful of unnecessary DynamoDB operations.

What format should our pre-aggregation step output? Exactly the same as the format of our data in DynamoDB! The pre-aggregation step and the in-Dynamo aggregation differ from each other in terms of their scope (100 events versus all-history) and their storage mechanism (local in-memory versus remote database), but they represent the same analytics algorithm, and a different intermediate format would only confuse our OOPS colleagues. Table 11.5 reminds us of the format we need to populate in our Lambda function.

Table 11.5. Our DynamoDB row layout dictates the format of our pre-aggregated row in our Lambda function.

Truck VIN	Latitude	Longitude	Location timestamp	Mileage	Mileage at oil change
1HGCM8...	51.5...	-0.15...	2018-08-02T21:50:49Z	12453	12418

We should be ready now to create our first analytics-on-write code for this AWS Lambda. Create the following file:

```
src/main/scala/aowlambda/aggregator.scala
```

This file should be populated with the contents of the following listing.

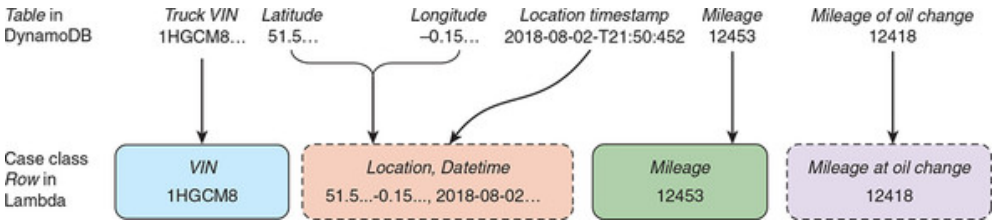
Listing 11.3. aggregator.scala

```
package aowlambda

import org.joda.time.DateTime, aowlambda.{TruckArrives => TA},
  aowlambda.{TruckDeparts => TD}, aowlambda.{MechanicChangesOil => MCO}

case class Row(vin: String, mileage: Int, mileageAtOilChange: Option[Int
  locationTs: Option[(Location, DateTime)])

object Aggregator {

  def map(event: Event): Option[Row] = event match {
    case TA(ts, v, loc) => Some(Row(v.vin, v.mileage, None, Some(loc, ts
    case TD(ts, v, loc) => Some(Row(v.vin, v.mileage, None, Some(loc, ts
    case MCO(ts, _, v)  => Some(Row(v.vin, v.mileage, Some(v.mileage), N
    case _              => None
  }


  def reduce(events: List[Option[Row]]): List[Row] =
    events
      .collect { case Some(r) => r }
      .groupBy(_.vin)
      .values
      .toList
      .map(_.reduceLeft(merge))

  private val merge: (Row, Row) => Row = (a, b) => {

    val m = math.max(a.mileage, b.mileage)
    val maoc = (a.mileageAtOilChange, b.mileageAtOilChange) match {
      case (l @ Some(_), None) => l
      case (l @ Some(lMaoc), Some(rMaoc)) if lMaoc > rMaoc => l
      case (_, r) => r
    }
    val locTs = (a.locationTs, b.locationTs) match {
      case (l @ Some(_), None) => l
      case (l @ Some((_, lTs)), Some((_, rTs))) if lTs.isAfter(rTs) => l
      case (_, r) => r
    }
    Row(a.vin, m, maoc, locTs)
  }
}
```

- ***1*** **The intermediate format for a truck's metrics**
- ***2*** **Transforms an event into the intermediate format**
- ***3*** **Reduces all events into one row per truck**
- ***4*** **Consolidates two rows into one by taking the newer/higher values**

There's quite a lot to unpack in our aggregator.scala file; let's make sure you understand what it's doing before moving on. First, we have the imports, including aliases for our event types to make the subsequent code fit on one line. Then we introduce a case class called `Row`; this is a lightly adapted version of the data held in table 11.5. Figure 11.10 shows the relationship between table 11.5 and our new `Row` case class; we will be using instances of this `Row` to drive our updates to DynamoDB.

Figure 11.10. The `Row` case class in our Lambda function will contain the same data points as found in our DynamoDB table. The dotted lines indicate that a given `Row` instance may not contain these data points, because the related event types were not found in this microbatch for this OOPS truck.



Moving onto our `Aggregator` object, the first function we see is called `map`. This function transforms any incoming OOPS event into either `Some(Row)` or `None`, depending on the event type. We use a Scala pattern match on the event type to determine how to transform the event:

- The three event types with relevant data for our analytics are used to populate different slots in a new `Row` instance.
- Any other event types simply output a `None`, which will be filtered out later.

If you are unfamiliar with the `Option` boxing of the `Row`, you can find more on these techniques in chapter 8.

The second public function in the `Aggregator` is called `reduce`. This takes a list of `Option`-boxed `Rows` and squashes them down into a hopefully smaller list of `Rows`. It does this by doing the following:

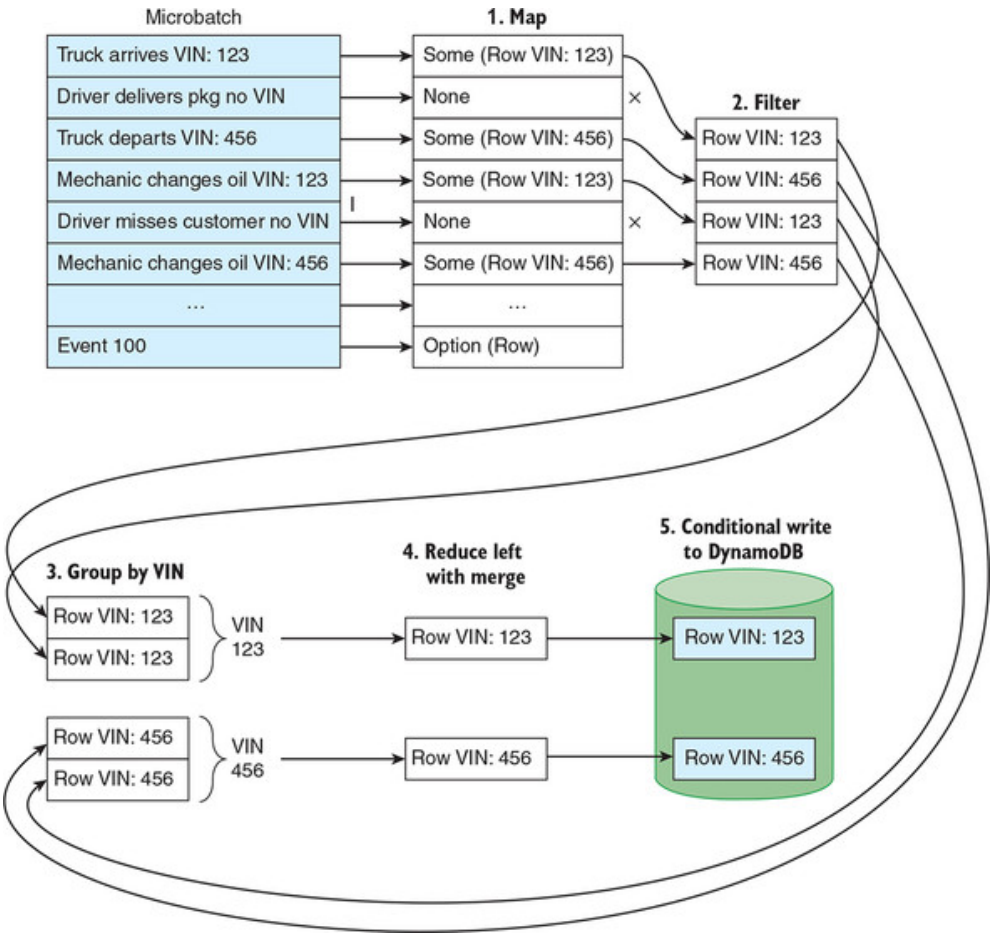
- Filtering out any `Nones` from the list
- Grouping the `Rows` by the VIN of the truck described in that `Row`
- Squashing down each group of `Rows` for a given truck into a single `Row` representing every potential update from the microbatch for this truck

Phew! If this seems complicated, it's because it *is* complicated. We are rolling our own map-reduce algorithm to run against each microbatch of events in this Lambda. With the sophisticated query languages offered by

technologies such as Apache Hive, Amazon Redshift, and Apache Spark, it's easy to forget that, until relatively recently, coding MapReduce algorithms for Hadoop in Java was state-of-the-art. You could say that with our Lambda, we are getting back to basics in writing our own bespoke map-reduce code.

How do we squash a group of `Rows` for a given truck down to a single `Row`? This is handled by our `reduceLeft`, which takes pairs of `Rows` and applies our `merge` function to each pair iteratively until only one `Row` is left. The `merge` function takes two `Rows` and outputs a single `Row`, combining the most recent data-points from each source `Row`. Remember, the whole point of this pre-aggregation is to minimize the number of writes to DynamoDB. Figure 11.11 shows the end-to-end map-reduce flow.

Figure 11.11. First, we map the events from our microbatch into possible `Rows`, and filter out the `Nones`. We then group our `Rows` by the truck's VIN and reduce each group to a single `Row` per truck by using a merge. Finally, we perform a conditional write for each `Row` against our table in DynamoDB.

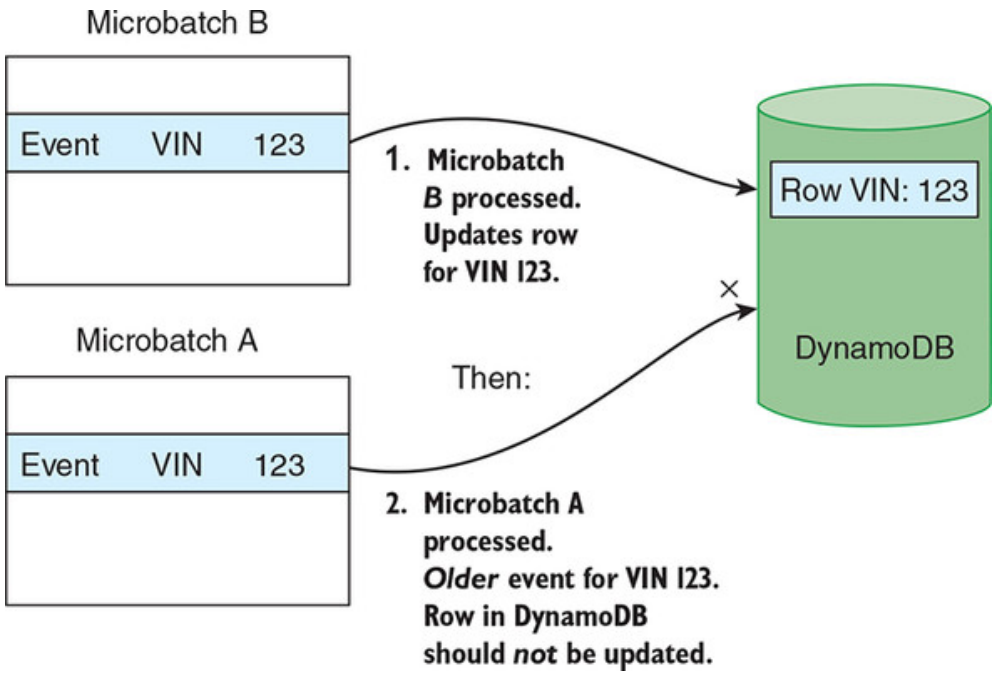


That completes our map-reduce code for pre-aggregating our microbatch of events prior to updating DynamoDB. In the next section, let's finally write that DynamoDB update code.

11.2.5. Conditional writes to DynamoDB

Thanks to our `merge` function, we know that the reduced list of `Rows` represents the most recent data-points for each truck *from within the current microbatch*. But it is possible that another microbatch with more recent events for one of these OOPS trucks has already been processed. Event transmission and collection is notoriously unreliable, and it's possible that our current microbatch contains old events that got delayed somehow—perhaps because an OOPS truck was in a road tunnel, or because the network was down in an OOPS garage. Figure 11.12 illustrates this risk of our microbatches being processed out of order.

Figure 11.12. When our Lambda function processes events for OOPS Truck 123 out of chronological order, it is important that a more recent data point in DynamoDB is not overwritten with a stale data point.

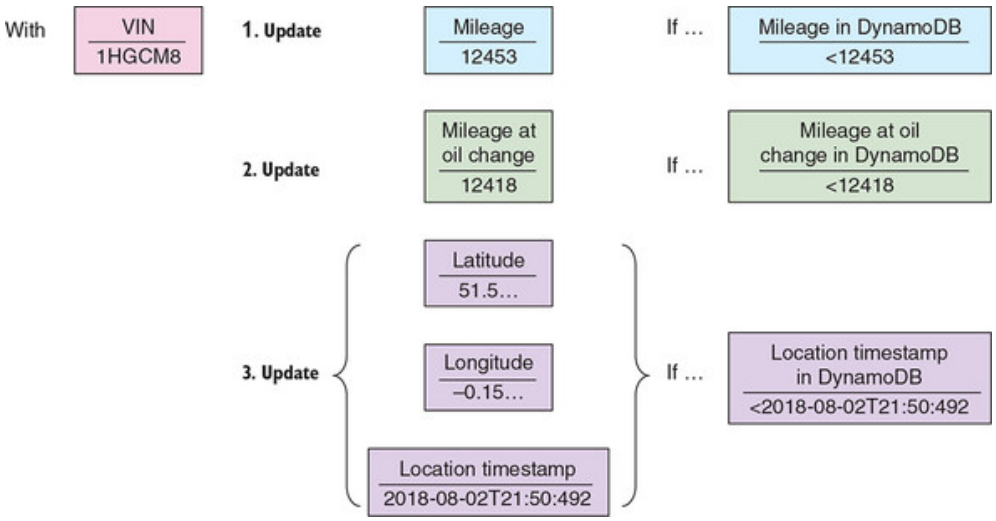


As a result, we cannot blindly overwrite the existing row in DynamoDB with the new row generated by our Lambda function. We could easily overwrite a more recent truck mileage or location with an older one! The solution has been mentioned briefly in previous sections of this chapter; we are going to use a feature of DynamoDB called *conditional writes*.

When I alluded to conditional writes earlier in this chapter, I suggested that there was some kind of read-check-write loop performed by the Lambda against DynamoDB. In fact, things are simpler than this: all we have to do in our Lambda is send each write request to DynamoDB with a condition attached to it; DynamoDB will then check the condition against the current state of the database, and apply the write only if the condition passes. Figure 11.13 shows the set of conditional writes our Lambda will attempt to perform for each row.

Figure 11.13. For each `Row`, we attempt three writes against DynamoDB, where each write is dependent on a condition in

DynamoDB passing.



We are ready to take the logic from figure 11.13 and implement it in Scala. For this, we will use the AWS Java SDK, which includes a DynamoDB client. To keep our code succinct, we will also use the AWScala project, which is a more Scala-idiomatic domain-specific language (DSL) for working with DynamoDB.

Create this Scala file:

```
src/main/scala/aowlambda/Writer.scala
```

This file should be populated with the contents of the following listing.

Listing 11.4. Writer.scala

```
package aowlambda

import awscala._, dynamodbv2.{AttributeValue => AttrVal, _}
import com.amazonaws.services.dynamodbv2.model._
import scala.collection.JavaConverters._

object Writer {
  private val ddb = DynamoDB.at(Region.US_EAST_1)

  private def updateIf(key: AttributeValue, updExpr: String,
    condExpr: String, values: Map[String, AttributeValue],
    names: Map[String, String]) {                                    1

    val updateRequest = new UpdateItemRequest()
      .withTableName("oops-trucks")
      .addKeyEntry("vin", key)
      .withUpdateExpression(updExpr)
      .withConditionExpression(condExpr)
      .withExpressionAttributeValues(values.asJava)
      .withExpressionAttributeNames(names.asJava)
```

```
      try {
        ddb.updateItem(updateRequest)
      } catch { case ccfe: ConditionalCheckFailedException => }      // 2
    }

    def conditionalWrite(row: Row) {
      val vin = AttrVal.toJavaValue(row.vin)

      updateIf(vin, "SET #m = :m",
        "attribute_not_exists(#m) OR #m < :m",
        Map(":m" -> AttrVal.toJavaValue(row.mileage)),
        Map("#m" -> "mileage"))                                      // 3

      for (maoc <- row.mileageAtOilChange) {                         // 4
        updateIf(vin, "SET #maoc = :maoc",
          "attribute_not_exists(#maoc) OR #maoc < :maoc",
          Map(":maoc" -> AttrVal.toJavaValue(maoc)),
          Map("#maoc" -> "mileage-at-oil-change"))
      }

      for ((loc, ts) <- row.locationTs) {                            // 5
        updateIf(vin, "SET #ts = :ts, #lat = :lat, #long = :long",
          "attribute_not_exists(#ts) OR #ts < :ts",
          Map(":ts"   -> AttrVal.toJavaValue(ts.toString),
              ":lat"  -> AttrVal.toJavaValue(loc.latitude),
              ":long" -> AttrVal.toJavaValue(loc.longitude)),
          Map("#ts"   -> "location-timestamp", "#lat" -> "latitude",
              "#long" -> "longitude"))
      }
    }
  }
```

- ***1* Perform a conditional write using the supplied arguments.**
- ***2* If the condition check fails, an Exception is thrown, which we will silently swallow.**
- ***3* Update the truck's mileage if the truck's current mileage is missing or lower than the new value.**
- ***4* Update the miles at oil change if it is supplied and the figure in DynamoDB is lower.**
- ***5* Update the three location fields if they are supplied and the timestamp in DynamoDB is lower.**

The code for interfacing with DynamoDB is admittedly verbose, but squint at this code a little and you can see that it follows the basic shape of figure 11.13. We have a `Writer` module that exposes a single method, `conditionalWrite`, which takes in a `Row` as its argument. The method does not return anything; it exists only for the side effects it performs, which are conditional writes of the three elements of the `Row` against the

same truck's row in DynamoDB. These conditional writes are performed using the private `updateIf` function, which helps construct an `UpdateItemRequest` for the row in DynamoDB.

A few things to note about this code:

- The `updateIf` function takes the truck's VIN, the update statement, the update condition (which must pass for the update to be performed), the values required for the update, and attribute aliases for the update.
- The update statement and update condition are written in DynamoDB's custom expression language,[1] which uses `:some-value` for attribute values and `#some-alias` for attribute aliases.

  1

  For more information on Dynamo DB's expression language, refer to https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Expressions.html.

- For mileage at oil change and the truck location, those conditional writes are themselves dependent on the corresponding parts of the `Row` being populated.
- For brevity, we do not have any logging or error handling. In a production Lambda function, we would include both.

That completes our DynamoDB-specific code!

11.2.6. Finalizing our Lambda

Now we are ready to pull our logic together into the Lambda function definition itself. Create our fourth and final Scala file:

```
src/main/scala/aowlambda/LambdaFunction.scala
```

This file should be populated with the contents of the following listing.

Listing 11.5. LambdaFunction.scala

```
package aowlambda

import com.amazonaws.services.lambda.runtime.events.KinesisEvent
import scala.collection.JavaConverters._

class LambdaFunction {

  def recordHandler(microBatch: KinesisEvent) {

    val allRows = for {
```

```
        recs <- microBatch.getRecords.asScala.toList
        bytes = recs.getKinesis.getData.array
        event = Event.fromBytes(bytes)
        row = Aggregator.map(event)
      } yield row

      val reducedRows = Aggregator.reduce(allRows)        2

      for (row <- reducedRows) {                          3
        Writer.conditionalWrite(row)
      }
    }
  }
```

- ***1*** **Converts our microbatch of events into a List of Rows ready for reduction**
- ***2*** **Reduces our Rows to the minimal set of possible updates for DynamoDB**
- ***3*** **Loops through each Row and performs a conditional write in DynamoDB**

It is this function, `aowlambda.LambdaFunction.recordHandler`, that will be invoked by AWS Lambda for each incoming microbatch of Kinesis events. The code should be fairly simple:

1. We convert each Kinesis record in the microbatch into a `Row` instance.
2. We run our reduce function to pre-aggregate all of our `Rows` down to the minimal set of DynamoDB updates to attempt (see listing 11.3).
3. We loop through the remaining `Rows` and perform a DynamoDB conditional write for each (see listing 11.4).

Code complete! Let's quickly check that the code compiles fine, like so:

```
guest$ sbt compile
...
[info] Compiling 2 Scala sources to /vagrant/ch11/11.2/aow-
     lambda/target/scala-2.11/classes...
[success] Total time: 19 s, completed 20-Dec-2018 21:00:45
```

Now we need to assemble a fat jar with all of our Lambda function's dependencies in it. Let's use the `sbt-assembly` plugin for Scala Build Tool to do this:

```
guest$ sbt assembly
...
[info] Packaging /vagrant/ch11/11.2/aow-lambda/target/scala-2.11/
➥ aow-lambda-0.1.0 ...
```

```
[info] Done packaging.
[success] Total time: 516 s, completed 20-Dec-2018 21:10:04
```

Great—our build has now been completed. This completes the coding required for our Lambda function. In the next section, we will get this deployed and put it through its paces!

11.3. Running our Lambda function

Much of the appeal of a system like AWS Lambda lies in its "serverless" promise. Of course, servers *are* involved in running our Lambda, but they are hidden from us, with AWS taking responsibility for operating our function reliably. The price we pay for outsourcing our ops is a relatively involved setup process, which we will go through now.

11.3.1. Deploying our Lambda function

Earlier in this chapter, we set up our Kinesis stream and our DynamoDB table. We now need to deploy our assembled Lambda function and wire it into our stream and table. We will perform all of this by using the AWS CLI tools. There is a lot of ceremony to step through here, so let's get started.

Uploading to S3

First, we need to make our local fat jar accessible to the AWS Lambda service. We do this by uploading the file to Amazon S3. This is straightforward using the AWS CLI. First, we create the bucket:

```
$ s3_bucket=ulp-ch11-fatjar-${your_first_pets_name}
$ jar=aow-lambda-0.1.0
$ aws s3 mb s3://${s3_bucket} --profile=ulp --region=us-east-1
make_bucket: s3://ulp-ch11-fatjar-little-torty/
```

And next we upload the jar file to S3:

```
$ aws s3 cp ./target/scala-2.12/${jar} s3://${s3_bucket}/ --profile=ulp
upload: target/scala-2.12/aow-lambda-0.1.0 to
 s3://ulp-ch11-fatjar-little-torty/aow-lambda-0.1.0
```

Unfortunately, there is a bug with AWS whereby uploading large files to newly created buckets can hang. If that happens, take a break and try again in an hour or so; you can continue with the rest of the setup in the meantime. Next, we need to configure the necessary permissions for our Lambda function.

Configuring permissions

The permissions required for operating a Lambda function are complex, so we are going to take a shortcut by using a CloudFormation template that we prepared earlier. AWS CloudFormation is a service that lets you spin up various AWS resources by using JSON templates; you can think of a CloudFormation template as the declarative JSON recipe for creating a collection of AWS services configured exactly as you want them. In Amazon parlance, we will use this template to create a *stack* of AWS resources —in our case, IAM roles to operate our Lambda function.

The pre-prepared CloudFormation template is publicly available in S3 in the `ulp-assets` bucket:

```
$ template=https://ulp-assets.s3.amazonaws.com/ch11/cf/aow-lambda.templa
```

Kick off the stack creation like so:

```
$ aws cloudformation create-stack --stack-name AowLambda \
  --template-url ${template} --capabilities CAPABILITY_IAM \
  --profile=ulp --region=us-east-1
{
    "StackId": "arn:aws:cloudformation:us-east-1:719197435995:
➥ stack/AowLambda/392e05e0-5963-11e5-aa74-5001ba48c2d2"
}
```

You can monitor Amazon's progress in creating this stack by using this command:

```
$ aws cloudformation describe-stacks --stack-name AowLambda \
  --profile=ulp --region=us-east-1
```

When you see a `StackStatus` of `CREATE_COMPLETE` in the returned JSON, we are ready to continue. We will need another piece of this output in the next section: the `Output-Value` for the `ExecutionRole`, which should start with `arn:aws:iam::`. Let's create a new environment variable set to this value:

```
$ role_arn="arn:aws:iam::719197435995:role/AowLambda-LambdaExecRole
➥ -1CNLT4WVY6PN4"
```

When creating this variable, make sure not to include any line breaks in the value.

Creating our Lambda function

The next step is to register our function with AWS Lambda. We can do this with a single AWS CLI command:

```
$ aws lambda create-function --function-name AowLambda \
  --role ${role_arn} --code S3Bucket=${s3_bucket},S3Key=${jar} \
  --handler aowlambda.LambdaFunction::recordHandler \
  --runtime java8 --timeout 60 --memory-size 1024 \
  --profile=ulp --region=us-east-1
   {
     "FunctionName": "AowLambda",
     "FunctionArn": "arn:aws:lambda:us-east-1:089010284850:function:
➥ AowLambda",
     "Runtime": "java8",
     "Role": "arn:aws:iam::089010284850:role/AowLambda-LambdaExecRole
➥ -FUVSBSEC1Y6R",
     "Handler": "aowlambda.LambdaFunction::recordHandler",
     "CodeSize": 27765196,
     "Description": "",
     "Timeout": 60,
     "MemorySize": 1024,
     "LastModified": "2018-12-21T07:44:31.073+0000",
     "CodeSha256": "jRpr4E60rP4hznB1Q/ApO6+fOAnLHMwfyhhT3rU5KWM=",
     "Version": "$LATEST",
     "TracingConfig": {
        "Mode": "PassThrough"
     },
     "RevisionId": "0609f559-fd1f-45c9-aee2-11ee159183b5"
   }
```

This command defines a function called `AowLambda` by using our IAM role and our fat jar previously uploaded to S3. The `--handler` argument tells AWS Lambda exactly which method to invoke inside our fat jar. The next three arguments configure the exact operation of the function: the function should be run against Java 8 (as opposed to Node.js), should time out after 60 seconds, and should be given 1 GB of RAM.

Attaching our function to Kinesis

Are we there yet? Not quite. We still need to identify the Kinesis stream we created earlier as the event source for our Lambda function. We do this by creating an *event source mapping* between the Kinesis stream and the function, again using the AWS CLI:

```
$ aws lambda create-event-source-mapping \
  --event-source-arn ${stream_arn} \
  --function-name AowLambda --enabled --batch-size 100 \
  --starting-position TRIM_HORIZON --profile=ulp --region=us-east-1
{
    "UUID": "bdf15c0b-a565-4a15-b790-6c2247d9aba3",
```

```
    "StateTransitionReason": "User action",
    "LastModified": 1545378677.527,
    "BatchSize": 100,
    "EventSourceArn": "arn:aws:kinesis:us-east-1:719197435995:stream/
➥ oops-events",
    "FunctionArn": "arn:aws:lambda:us-east-
     1:719197435995:function:AowLambda",
    "State": "Creating",
    "LastProcessingResult": "No records processed"
}
```

From the returned JSON, you can see that we have successfully connected our Lambda function to our stream of OOPS events. It reports that no records have been processed yet. In the next section, we will re-enable our event generator and check the results.

11.3.2. Testing our Lambda function

Remember that our generator script is available in the GitHub repository:

```
ch11/11.1/generate.py
```

Let's kick off our event generator:

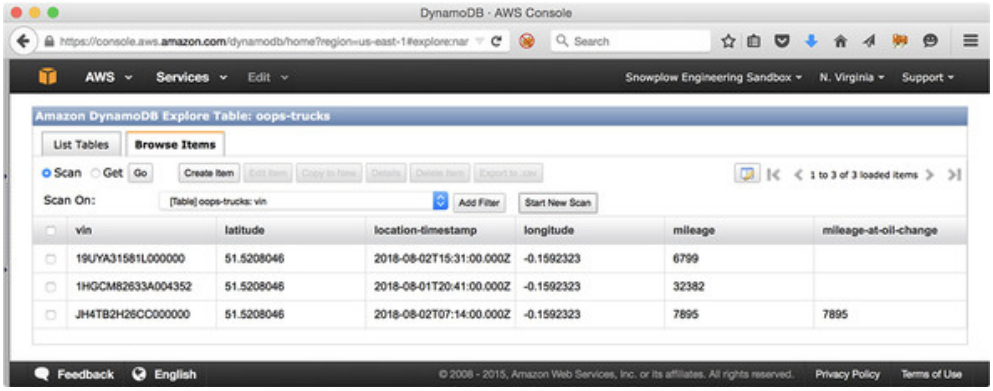
```
$ /vagrant/ch11/11.1/generate.py
Wrote DriverDeliversPackage with timestamp 2018-01-01 02:31:00
Wrote DriverMissesCustomer with timestamp 2018-01-01 05:53:00
Wrote TruckDepartsEvent with timestamp 2018-01-01 08:21:00
```

Great! Those events are now being sent to our Kinesis stream. Let's now take a look at our DynamoDB table. From the AWS dashboard:

- Make sure you are in the N. Virginia region by using the top-right drop-down menu.
- Click DynamoDB.
- Click the `oops-truck` table entry and click the Explore Table button.

You should see three rows with all of the expected fields, as shown in figure 11.14.

Figure 11.14. Our DynamoDB table oops-trucks contains the six fields expected by the OOPS business intelligence team. Note that the Lambda has observed an oil change event for only one of the trucks so far.

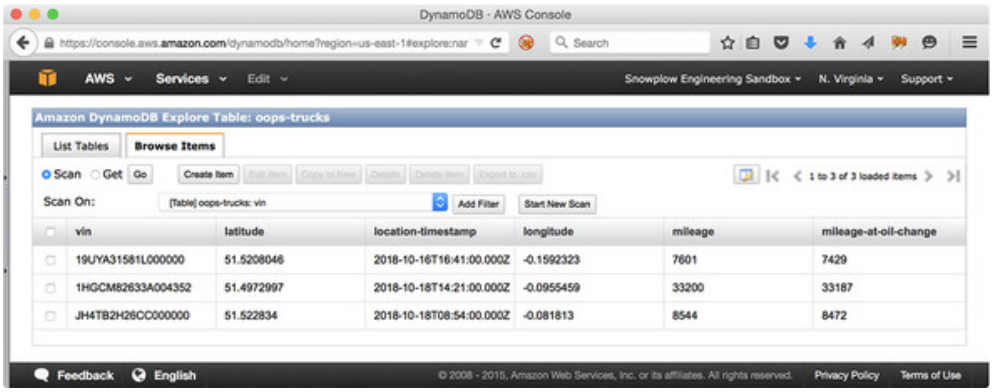


Leave our event generator running a little longer and then refresh the table in the DynamoDB interface. You should see the following:

- We now have values populated for all attributes for all three trucks. In fact, our generator sends only events for three OOPS trucks.
- Our location timestamps roughly match the most recent timestamps we see printed in the terminal running our generator, showing that our Lambda function is keeping up.
- The trucks' latitude and longitude have changed. Expect to see the same latitudes and longitudes repeated frequently, as the generator uses only five locations.
- The mileage-at-oil-change for each truck lags the truck's total mileage.

Figure 11.15 shows this updated view we are expecting to see in DynamoDB.

Figure 11.15. We now have all values populated for each of our three trucks. These values are being constantly updated by our AWS Lambda function in response to new events.



So far, so good—we can see that our Lambda is working well to keep our operational dashboard in DynamoDB up-to-date with the latest stream of events from OOPS trucks.

The last thing we want to check is that our conditional writes are correctly guarding against old events that could arrive out of order. Remember that we don't want to overwrite a fresh data point in DynamoDB with older data just because the older event arrives after the newer event.

To test this, first refresh the DynamoDB table to get the latest data. Then switch back to your generator's terminal window and press Ctrl-C:

```
Wrote TruckArrivesEvent with timestamp 2018-03-06 00:39:00
^CTraceback (most recent call last):
  File "./generate.py", line 168, in <module>
    time.sleep(1)
KeyboardInterrupt
```

Finally, we restart the generator with a new command-line argument, backward:

```
$ ./generate.py backwards
Wrote DriverDeliversPackage with timestamp 2017-12-31 20:15:00
Wrote TruckArrivesEvent with timestamp 2017-12-31 18:40:00
Wrote MechanicChangesOil with timestamp 2017-12-31 15:45:00
```

As you can see, the generator is now working backward, creating ever-older events. Leave it a short while, and then head back into the DynamoDB interface and refresh the table. You'll see that all of the data points are unchanged: the old events are not impacting on our latest truck statuses in DynamoDB at all.

For a final test, leave the backward-stepping event generator running, open a new terminal, and kick off the forward-stepping generator again:

```
$ ./generate.py
Wrote DriverDeliversPackage with timestamp 2018-01-01 00:38:00
Wrote TruckArrivesEvent with timestamp 2018-01-01 05:03:00
Wrote MechanicChangesOil with timestamp 2018-01-01 08:41:00
```

You will have to wait a while until the event timestamps that are generated overtake your current location-timestamp values in DynamoDB, but after this happens, you should see your truck data points in DynamoDB starting to refresh again.

And that completes our testing! Hopefully, our colleagues at OOPS are pleased with the result: an analytics-on-write system that can keep a dashboard of OOPS trucks updated in near real-time from a Kinesis stream. A nice evolution for this project would be to add a visualization layer on top of the DynamoDB table, perhaps implemented in D3.js or a similar library.

Summary

- With analytics-on-write, we decide on the analysis we want to perform ahead of time, and then put this analysis live on our event stream.
- Analytics-on-write is good for low-latency operational reporting and dashboards to support thousands of simultaneous users. Analytics-on-write is a complement to analytics-on-read, not a competitor.
- To support the latency and access requirements, analytics-on-write systems typically lean heavily on horizontally scalable key-value stores such as Amazon DynamoDB.
- Applying analytics-on-write retrospectively can be difficult, so it is important to gather requirements and agree on the proposed algorithm up-front.
- We implemented a simple truck status dashboard for OOPS by using AWS Lambda as the stream processing framework reading OOPS events from Amazon Kinesis.
- To reduce the number of calls to DynamoDB, we implemented a local map-reduce on the microbatch of events received by the Lambda function, reducing this batch of 100 events down to the minimal number of potential updates to DynamoDB.
- AWS Lambda is essentially stateless, so we used DynamoDB's conditional writes to ensure that we were always updating our DynamoDB table with the latest data points relating to each OOPS truck.
- There was a close mapping between the dashboard required by OOPS, the layout of the table in DynamoDB, and the Scala representation of a row used in our Lambda's local map-reduce.